



ESTADO A 1 DE SEPTIEMBRE DE 2026

Estudio de batería

Qué está hecho, dónde vive cada cosa y cómo se abre.

FUNCIONANDO

El motor de optimización, las doce tablas, la curva de precio a veinte años, el API y la pantalla conectada a él.

A MEDIAS

La pantalla lee la curva real del servidor, pero el despacho que dibuja todavía lo calcula el navegador.

SIN EMPEZAR

La identificación del usuario, la búsqueda del tamaño óptimo y probar sin subir ficheros.

Lo que se puede hacer hoy, de punta a punta

PASO	ESTADO	CON QUÉ
Subir curva de consumo y generación	Hecho	cargar_curvas.py · POST /api/bat/curvas
Validar que el fichero es lo que dice	Hecho	Cambio de hora, bisiestos, unidad, solar que no genere de noche
Publicar la curva de precio	Hecho	cron_diario.py · a diario a las 11:45
Optimizar la batería	Hecho	caso.py ejecutar · HiGHS, 20 escenarios
Guardar y consultar resultados	Hecho	app_case_run · _result_annual · _dispatch
Comprobar que la cadena es correcta	Hecho	comprobar.py --base · 18 comprobaciones
Verlo en una pantalla	A medias	Lee la curva real; el despacho lo calcula el navegador
Saber quién es el usuario	No	Llega un correo por parámetro y se cree

El resultado de la última pasada. Fábrica de 350 MWh al año con 250 kWp de fotovoltaica: la batería de **50 kW / 4 h** sale con un VAN de **+2.544 €** y positiva en el **100 % de los escenarios**. Las de 100 y 200 kW salen negativas. El diagnóstico está en los ciclos: la pequeña trabaja 1,0 al día y la grande 0,7.

Cómo se abre cada cosa

La pantalla del estudio

Se sirve **desde el propio API**, no como fichero suelto. Abierta con `file://` el navegador la trata como otro origen y bloquea todas las llamadas: se ve la página y no se ve un solo dato.

```
$ cd /home/ubuntu/scripts
$ uvicorn production.api.main:app --host 127.0.0.1 --port 8000
```

```
# y en el navegador:
http://127.0.0.1:8000/bateria/estudio_bateria_dev.html
```

RUTA	QUÉ ES
/bateria/estudio_bateria_dev.html	La buena. Mes navegable, balance de energía, detalle económico
/bateria/estudio_bateria.html	Versión mínima. Los resultados son fijos, solo enseña el flujo
/plantillas/plantilla_consumo.csv	Las cuatro plantillas de ejemplo, también en <code>.xlsx</code>
/api/docs	La documentación del API, generada sola
/	El panel de predicciones de siempre, intacto

El API

VERBO	RUTA	QUÉ HACE
GET	/api/bat/estado	Qué curva de precio hay publicada
GET	/api/bat/curva	Los percentiles P10/P50/P90 de un tramo. Máximo 400 días
GET	/api/bat/instalaciones	Lo que el usuario ya tiene guardado
POST	/api/bat/curvas	Subir consumo y generación
POST	/api/bat/estudio	Lanzar el cálculo. Devuelve un identificador de tarea
GET	/api/bat/estudio/{tarea}	Por qué escenario va, y el resultado al terminar
GET	/api/bat/despacho/{run_id}	El despacho hora a hora. Máximo 62 días

El estudio devuelve una tarea, no un resultado. Tarda unos cuatro minutos: veinte escenarios sobre 7.426 días. Eso no cabe en una petición HTTP — el navegador y el nginx de delante cortan mucho antes. El cálculo va por detrás y la pantalla pregunta cómo va.

Falta `python-multipart` en el servidor. Sin él, la subida de ficheros contesta un 501 y todo lo demás sigue funcionando: `pip install python-multipart`

Dónde vive cada dato

Doce tablas nuevas, todas con el prefijo `app_` para que se distingan de las del pipeline de datos. Se crean con `production/app/sql/crear_tablas_bess.sql`, que es idempotente.

TABLA	FILAS HOY	QUÉ GUARDA
QUIÉN		
<code>app_user</code>	1	Correo. Todo lo demás cuelga de aquí
LO QUE EL USUARIO APORTA		
<code>app_battery_model</code>	5	La ficha: potencia, duración, rendimiento, ciclos, precio
<code>app_consump_inst</code>	3	La instalación de consumo: MWh al año, peajes, potencia contratada
<code>app_consump_shape</code>	576 por curva	La forma horaria: 12 meses × laborable/finde × 24 h
<code>app_gen_inst</code>	4	La generación: tecnología, kWp, degradación, límite de vertido
<code>app_gen_shape</code>	576 por curva	Ídem para la generación
LO QUE PONE EL SISTEMA		
<code>app_curve</code>	1 · siempre 1	La curva de precio publicada. Un índice único impide que haya dos
<code>app_curve_hourly</code>	178.224	Los percentiles P10/P50/P90 hora a hora, hasta 2046
EL ESTUDIO Y SU RESULTADO		
<code>app_study_case</code>	7	El caso: qué batería, qué instalaciones, qué periodo, qué política
<code>app_case_run</code>	7	La tarjeta de resultado. VAN, ciclos, ahorro, cobertura del CAPEX
<code>app_case_result_annual</code>	21 por estudio	Año a año, con su banda de percentiles
<code>app_case_dispatch</code>	534.672 por estudio	Hora a hora: precio, carga, descarga, estado de carga, red

Dos cosas del esquema que hay que saber

La curva se pisa, no se acumula. Hay una fila en `app_curve` y un índice único sobre una expresión constante que impide que se cuele otra. Guardar una al día serían 65 millones de filas al año sin aportar nada: las anclas del escenario son medias del año en curso y entre dos días no se mueven. Por eso `app_case_run` **copia** la fecha y el hash de la matriz al ejecutar, en vez de referenciar una fila que va a cambiar.

El eje de horas va sin zona horaria. `app_curve_hourly.datetime` y `app_case_dispatch.datetime` son `TIMESTAMP` a secas, porque la curva vive en una rejilla nominal de 24 h/día. Con `TIMESTAMPZ`, el domingo de marzo en que las 02:00 no existen, Postgres mete las 02:00 y las 03:00 en la misma marca y revienta la clave primaria. `spot_price` y `predictions` sí son instantes reales y sí llevan zona.

Qué script usar para qué

FICHERO	PARA QUÉ
LA CURVA DE PRECIO · CORRE SOLA	
production/curva/cron_diario.py	La cadena diaria: matriz y después curva. Si la matriz falla, la curva no se república
production/curva/generar_curva.py	Publica el <code>.npy</code> y escribe la base
production/curva/ver_curva.py	Dibujarla sin base ni modelos, solo desde el artefacto
EL ESTUDIO	
production/app/cargar_curvas.py	Subir las dos curvas. Todo lo demás sale del fichero
production/app/cargar_perfil.py	El lector: tres formatos, cambio de hora, bisiestos, unidades
production/app/caso.py	Dar de alta, crear el caso y ejecutarlo
production/app/optimiza_bateria.py	El motor. Programación lineal con HIGHS
production/app/comprobar.py	Antes de fiarse de nada. 18 comprobaciones de la cadena entera
LA WEB	
production/api/main.py	El servidor. Sirve el API, la pantalla y las plantillas
production/api/bateria.py	Los siete endpoints del estudio
docs/web/estudio_bateria_dev.html	La pantalla. Fichero suelto, no toca nada de lo que ya hay

Un estudio completo, desde cero

```
$ export TFM_EMAIL="tu@correo"

# 1 · las curvas. El anual y los kWp se leen del fichero
$ python production/app/cargar_curvas.py \
  --consumo mi_consumo.xlsx --generacion mi_fv.csv

# 2 · la batería
$ python production/app/caso.py bateria --code BAT50 \
  --nombre "LFP 50 kW" --potencia 0.05 --duracion 4

# 3 · el caso y su ejecución
$ python production/app/caso.py crear --code EST1 --modo autoconsumo \
  --bateria BAT50 --consumo CONSUMO --generacion GENERACION \
  --desde 2027-01-01 --hasta 2046-12-31
$ python production/app/caso.py ejecutar --code EST1 --escenarios 20

# y para ver lo que hay
$ python production/app/caso.py listar
```

Antes de crearse un número, correr esto. Comprueba el esquema contra el código, que la curva cuadre con su artefacto, que el balance energético se cumpla hora a hora y que el óptimo coincida con dos casos resueltos a mano.

```
$ python production/app/comprobar.py --base
```

La conexión con el usuario

Todo está preparado para funcionar por usuario. Lo que no hay es forma de saber quién es.

Lo que ya está bien

Las seis tablas que el usuario alimenta cuelgan de `app_user` con `ON DELETE CASCADE`, y los códigos son únicos **por usuario**: dos personas pueden tener una instalación llamada `FABRICA` sin pisarse. Las consultas del API ya filtran por `user_id`. El modelo de datos no hay que tocarlo.

Lo que falta, y es un agujero

El usuario llega en un parámetro `email` y el servidor **se lo cree**. Ahora mismo, sabiendo el correo de alguien se pueden leer y escribir sus instalaciones. Para el TFM y la demo vale; para enseñárselo a nadie más, no.

QUÉ HACE FALTA	POR QUÉ
Login de verdad	Que el <code>user_id</code> salga de una sesión firmada, no de la URL
Cada consulta filtrada	Ya lo están, pero hoy filtran por lo que el cliente dice ser
Cuota por usuario	Cada estudio son cuatro minutos de CPU y medio millón de filas. Sin límite, uno solo puede tumbar el servicio
Limpieza de despachos	534.672 filas por estudio. Guardar todos para siempre no se sostiene: o se borran los viejos, o se guarda solo un escenario

La web de Pulso Energía ya tiene login. Lo razonable no es montar otro, sino que el API confíe en el que hay: que la sesión de la web llegue al servidor y de ahí salga el correo, en vez de que lo ponga el cliente.

Y lo demás que quedó apuntado

QUÉ	DÓNDE ESTÁ EL DETALLE
Buscar el tamaño óptimo	El VAN es unimodal, así que basta una búsqueda ternaria: diez resoluciones en vez de un barrido. <code>docs/web/pendiente.md</code>
Probar sin subir ficheros	Batería sola contra el mercado, o perfiles tipo. Con el aviso de que un perfil tipo no sirve para decidir una inversión
Que el despacho venga del servidor	Hoy la pantalla lo calcula en el navegador con una heurística. El endpoint <code>/api/bat/despacho</code> ya existe: falta que la pantalla lo use
El informe completo	El botón imprime la pantalla. El informe de verdad lo tiene que generar el servidor

Un aviso que no conviene perder. En las pruebas, una instalación fotovoltaica quedó cargada desde el fichero del **consumo**. Generaba 0,395 a medianoche. No falló nada: el estudio corrió y devolvió un VAN con dos decimales, describiendo un sitio que no existe. Con el perfil corregido la recomendación pasó de «*no sale a ninguna talla*» a «*sale, y en el 100 % de los escenarios*». Los casos `EST-01` y `DIM-50/100/200` —sin la R— están calculados con ese perfil malo y **no deben citarse**.